Theory of Convolutional Neural Network (CNN) using Fashion MNIST Dataset

Fashion Clothing Classification using CNN

This practical implements a **Convolutional Neural Network (CNN)** to classify fashion clothing images into different categories using the Fashion MNIST dataset.

The model predicts clothing categories such as:

- T-shirt
- Trouser
- Dress
- Shoe
- Bag
- Ankle boot etc.

---

1. What is CNN?

CNN stands for:

Convolutional Neural Network

CNN is a special type of Deep Learning model mainly used for:

- image classification
- object detection
- face recognition
- medical image analysis

CNN automatically learns:

- edges
- textures
- shapes
- patterns

from images.

---

2. Why CNN is Better for Images?

Traditional Neural Networks:

- flatten images into vectors
- lose spatial information

CNN:

- preserves image structure
- detects local patterns
- requires fewer parameters
- gives higher accuracy for images

---

3. Fashion MNIST Dataset

Fashion MNIST is an image dataset containing grayscale clothing images.

Dataset details:

Dataset	Images
Training Images	60,000
Testing Images	10,000

Image size:

28 × 28 pixels

Total classes:

10 classes

---

4. Fashion Categories

Label	Category
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

---

5. Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

NumPy

Used for:

- arrays
- numerical operations

Pandas

Used for dataframe handling.

Matplotlib

Used for:

- displaying images
- plotting graphs

TensorFlow

Used for:

- deep learning
- CNN model building

---

6. CNN Layers Imported

```
from tensorflow.keras.layers import:
Dense,
Conv2D,
MaxPooling2D,
Flatten
```

These are the major CNN building blocks.

---

7. Loading Dataset

```
fashion_mnist = tf.keras.datasets.fashion_mnist
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
```

Dataset is automatically downloaded from TensorFlow.

---

8. Shape of Dataset

```
print(X_train.shape)
```

Output:

(60000, 28, 28)

Meaning:

- 60,000 images
- each image size = 28×28

---

9. Displaying Image

```
plt.imshow(X_train[0])
plt.show()
```

Displays first training image.

Images are grayscale:

- black → low intensity
- white → high intensity

---

10. Data Normalization

```
X_train = X_train/255.0
X_test = X_test/255.0
```

Pixel values originally range from:

0 to 255

Normalization converts them into:

0 to 1

Why Normalization is Important?

Benefits:

- faster training

- stable gradients
- improved convergence
- better accuracy

---

11. Reshaping Data

X_train = X_train.reshape(60000,28,28,1)

X_test = X_test.reshape(10000,28,28,1)

CNN expects 4D input:

(samples, height, width, channels)

What is Channel?

- Grayscale image → 1 channel
- RGB image → 3 channels

Here:

1 channel

because images are grayscale.

---

12. Sequential Model

model = Sequential()

Sequential model means:

- layers are connected one after another.

---

13. Convolution Layer (Conv2D)

```
Conv2D(
    32,
    (3,3),
    activation='relu'
)
```

This is the most important CNN layer.

What is Convolution?

Convolution means:

applying filters/kernels to detect image features.

CNN learns:

- edges
- corners
- textures
- shapes

automatically.

Filters / Kernels

A filter slides over image and extracts patterns.

Example 3×3 filter:

\begin{bmatrix}1&0&-1\1&0&-1\1&0&-1\end{bmatrix}

This filter detects vertical edges.

Understanding Parameters

Conv2D(32,(3,3))

Parameter	Meaning
32	Number of filters
(3,3)	Filter size

Why Multiple Filters?

Different filters learn:

- edges
- curves
- textures
- clothing patterns

Feature Maps

After convolution:

- output image becomes feature map.

CNN extracts important visual information from image.

---

14. Activation Function — ReLU

activation='relu'

Formula:

$f(x)=\max(0,x)$

Purpose:

- introduces nonlinearity
- helps learn complex patterns
- avoids vanishing gradients

---

15. Max Pooling Layer

MaxPooling2D(pool_size=(2,2))

Pooling reduces image size.

How Max Pooling Works?

Example:

Original matrix:

$\begin{bmatrix}1&3\2&4\end{bmatrix}$

Max pooling output:

4

Largest value selected.

Advantages of Pooling

Advantage	Explanation
Reduces computation	Smaller feature maps
Prevents overfitting	Removes unnecessary details
Faster training	Fewer parameters
Feature robustness	Maintains important features

---

16. Flatten Layer

Flatten()

CNN outputs multidimensional feature maps.

Flatten converts them into a 1D vector.

Example:

28×28×32 → 25088 values

This vector is fed into Dense layers.

---

17. Dense Layer

Dense(128,activation='relu')

Dense layer performs final learning and classification.

128 neurons learn high-level patterns from extracted features.

---

18. Output Layer

Dense(10,activation='softmax')

Why 10 neurons?

- because dataset has 10 clothing categories.

Softmax Activation

Softmax converts outputs into probabilities.

Formula:

P(y_i)=\frac{e^{z_i}}{\sum_{j=1}^{n}e^{z_j}}

Output example:

Class	Probability
Shirt	0.01
Sneaker	0.02
Ankle boot	0.95

Highest probability becomes prediction.

---

19. Model Compilation

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Optimizer — Adam

Adam optimizer:

- updates weights
- minimizes loss
- improves predictions

Advantages:

- fast
- adaptive learning rate
- stable training

Loss Function

loss='sparse_categorical_crossentropy'

Used for multiclass classification with integer labels.

Formula:

$$L=-\log(P(y_{true}))$$

Lower loss means better predictions.

Accuracy Metric

Formula:

$$Accuracy=\frac{Correct\ Predictions}{Total\ Predictions}$$

---

20. Model Training

```
model.fit(
    X_train,
```

```
    y_train,
    epochs=5
)
```

Epoch

One complete pass through training dataset.

You trained:

5 epochs

Training Observation

Epoch	Accuracy
1	74%
5	93%

This shows:

- model learned successfully
- predictions improved gradually

---

21. Model Evaluation

loss, accuracy = model.evaluate(X_test,y_test)

Output:

Accuracy ≈ 91%

Meaning:

Model correctly classifies around 91 images out of 100.

---

22. Prediction

prediction = model.predict(X_test)

Returns probabilities for all 10 classes.

---

np.argmax()

predicted_class = np.argmax(prediction[0])

Returns class with highest probability.

Output:

9

Class Labels

class_names = [...]

Maps class numbers to clothing names.

Example:

Class	Name
9	Ankle boot

Final Prediction

print(class_names[predicted_class])

Output:

Ankle boot

---

23. Display Predicted Image

plt.imshow(X_test[0].reshape(28,28))

plt.show()

Displays the test image predicted by model.

---

24. Complete CNN Workflow

Load Dataset

↓

Normalize Images

↓

Reshape Data

↓

Apply Convolution

↓

Apply ReLU

↓

Apply Pooling

↓

Flatten Features

↓

Dense Layer Learning

↓

Softmax Classification

↓

Prediction Output

---

25. Why CNN Performs Well?

CNN:

- automatically extracts image features
- preserves spatial information
- reduces parameter count
- learns visual patterns effectively

This makes CNN highly suitable for image classification.

---

26. Advantages of CNN

Advantage	Explanation
High image accuracy	Learns visual patterns
Automatic feature extraction	No manual features needed
Parameter sharing	Efficient learning
Translation invariance	Detects objects anywhere

---

27. Conclusion

In this practical:

- Fashion MNIST dataset was used
- Images were normalized and reshaped

- CNN model built using TensorFlow/Keras
- Conv2D extracted image features
- MaxPooling reduced dimensions
- Dense layers performed classification
- Softmax predicted clothing categories
- Model achieved around 91% test accuracy

This practical demonstrates how CNNs are highly effective for image classification problems.